Diseño Técnico: StaticWebCatalog

Resumen ejecutivo

StaticWebCatalog es un editor de catálogo de productos que evoluciona en tres etapas: un prototipo local para validar la interfaz, una publicación estática con S3 para hacerlo visible, y una arquitectura serverless completa para producción multiusuario. Este documento explica las decisiones técnicas de cada etapa, las alternativas consideradas, y las razones detrás de cada elección.

Etapa 1: Prototipo local

Objetivo

Validar la interfaz de usuario: la grilla de productos, el encuadre de las imágenes, el flujo de decisión de precio, y la experiencia de carga. El prototipo corre enteramente en el navegador del autor, sin backend ni base de datos, y persiste hasta 10 productos en el almacenamiento local.

Decisión central: localStorage con data URI

El prototipo podría haber usado IndexedDB, pero con 10 productos y datos deliberadamente descartables, localStorage alcanza:

`10 productos × ~107 KB (data URI de JPEG 800px) ≈ 1.1 MB Presupuesto de localStorage ≈ 4.5 MB ———————— margen 4×`

IndexedDB sería correcto para producción, pero la diferencia de implementación no se justifica cuando el objetivo es validar apariencia y el contenido se va a tirar.

Alternativas consideradas

Almacenamiento	Pros	Contras
localStorage + b64	Trivial, sincrónico, sin configuración	No escala a 100 productos, pierde el original
IndexedDB	Escala a cientos, guarda originales	API async, más código, adelanta trabajo
Origin Private FS	Cuota similar a IndexedDB	Soporte desigual, sin ventaja real
Object URL persistido	Sin inflación base64	Muere al recargar: es un puntero al documento

La elección de localStorage es deliberadamente técnica para el alcance del prototipo. La deuda se hace explícita: **el data URI escalado es la única copia de la imagen**. Con 100 productos reales eso sería inaceptable; con 10 de prueba es el costo correcto.

Costura de repositorio: la decisión que amortigua el cambio

Aunque localStorage es sincrónico, toda lectura y escritura pasa por una interfaz de repositorio async:

```
js interface ProductRepository { list(): Promise<Product[]> add(product: Product, image: string): Promise<void> rename(id: string, name: string): Promise<void> remove(id: string): Promise<void> count(): Promise<number> }
```

Esta capa permite sustituir LocalStorageRepository por IndexedDbRepository o ApiRepository sin tocar las vistas. Las cuatro palabras clave `async/await` que cuesta hoy evitan el único retrofit caro: convertir un árbol de vistas síncronas a asíncronas.

Modelo de producto

```
js Product { id: string // uuid generado con crypto.randomUUID() name: string // editable, no vacío priceMode: 'amount' | 'inquire' priceCOP: number | null // entero ≥ 0 si 'amount'; null si 'inquire' image: string // data URI en el prototipo createdAt: number // epoch ms, orden de la grilla }
```

La invariante `priceMode === 'amount' ⟺ priceCOP` es un entero no negativo garantiza que "sin precio" sea una decisión explícita, no un olvido. Esto es importante en un catálogo de venta: un producto sin precio por descuido se lee como un error; uno que dice "Consultar precio" se lee como una decisión.

Encuadre sin recorte

Un catálogo mezcla orientaciones. Las dos opciones de `object-fit` fallan de maneras distintas:

Opción	Comportamiento	Problema
cover	Recorta para llenar el contenedor	Mutila productos verticales (vestidos, personas)
contain	Muestra la imagen completa con aire	La grilla se ve irregular

Se elige contain con fondo neutro: recortar el producto que se está vendiendo es peor que un poco de aire. La proporción del contenedor es 4:5, la habitual en catálogos de venta.

Decisión de precio explícita

Cada fila de la zona de pendientes exige una decisión antes de poder confirmarse: un monto en COP, o la marca "Consultar precio". El precio es opcional, pero la decisión no. Esto se traduce en dos estados de la fila:

- **Sin resolver**: no se puede confirmar, se señala como pendiente.
- **Resuelto**: tiene un monto o está marcado como "consultar".

La confirmación es parcial: solo las filas resueltas se agregan al catálogo; las demás permanecen con su estado intacto.

Límites y advertencias

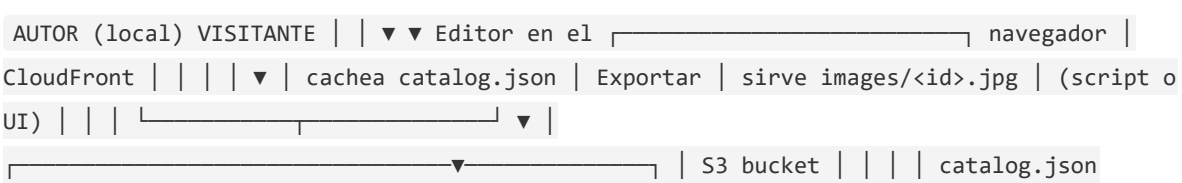
- **Máximo 10 productos**: validado en el repositorio, no en la vista.
- **Datos descartables**: se advierte que el contenido es de prueba y vive solo en este navegador.
- **Advertencia al salir**: con filas pendientes, el navegador solicita confirmación antes de recargar.

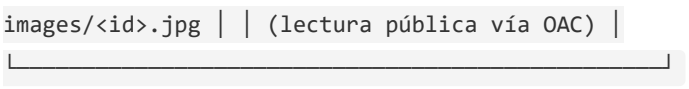
Etapa 2: Publicación estática con S3

Objetivo

Hacer el catálogo visible para terceros. El autor sigue editando localmente, pero puede exportar un `catalog.json` y las imágenes a un bucket S3 público. El visitante lee desde CloudFront.

Arquitectura





La escritura sigue siendo local. La publicación es un acto deliberado del autor: exporta y sube.

Pregunta abierta: mecanismo de publicación

Opción	Descripción	Trade-off
Exportar + subir manual	El editor genera un zip, el autor lo sube	Sin credenciales expuestas, más pasos
CLI de publicación	Script local que sube con AWS CLI	Requiere configuración de creds
Subida desde el browser	El editor sube directo con Cognito Identity	Anticipa la Etapa 3

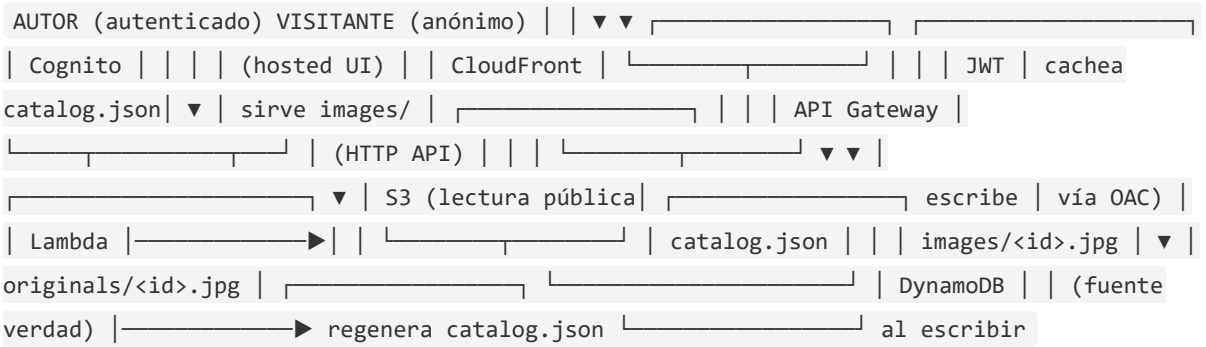
La Etapa 2 puede quedar en "exportar + subir manual" sin perder el objetivo de visibilidad.

Etapa 3: Arquitectura serverless

Objetivo

Producción completa: el autor edita directamente contra la nube, la autenticación protege la escritura, y la infraestructura escala sin mantenimiento de servidores.

Arquitectura



Separación de lectura y escritura

Un catálogo se lee mucho más de lo que se escribe. Por eso el camino de lectura no pasa por cómputo:

Ruta	Componentes	Costo por visita
Escritura	Cognito → API GW → Lambda → DynamoDB	Pago por request
Lectura	CloudFront → S3	Prácticamente cero

Cada escritura regenera `catalog.json` e invalida la caché de CloudFront. El visitante siempre ve la versión más reciente, sin cold starts ni costos de cómputo.

URLs prefirmadas para imágenes

Los bytes de las imágenes no pasan por Lambda:

`

1. POST /uploads → Lambda genera URL prefirmada + key
2. PUT directo a S3 ← El navegador sube, Lambda no ve los bytes
3. POST /products → {key, name, priceMode, priceCOP} Lambda escribe DynamoDB y regenera catalog.json

`

Esto evita el límite de 6 MB de payload de Lambda y el costo de cómputo por mover bytes.

Autenticación con Cognito

El endpoint de escritura no puede quedar abierto: cualquiera podría reemplazar el catálogo. Cognito con hosted UI evita construir pantallas de login:

- Un solo pool de usuarios.
- Login con email/contraseña o proveedores sociales.
- El frontend recibe un JWT que envía en el header `Authorization`.
- API Gateway valida el token antes de llegar a Lambda.

Fuente de verdad: DynamoDB

DynamoDB guarda el estado completo del catálogo. `catalog.json` es contenido derivado que se regenera con cada escritura. Esto permite:

- Escrituras atómicas por producto.
- Consultas por `createdAt` sin reconstruir nada.
- Escalar a múltiples autores sin conflictos.

A este volumen de productos, DynamoDB entra en la capa gratuita.

Preservación del original

A diferencia del prototipo, la Etapa 3 guarda dos versiones de cada imagen:

- `images/<id>.jpg` : imagen escalada a 800px para el catálogo.
- `originals/<id>.jpg` : archivo original para futuros usos (zoom, descarga, reescaldos).

El escalado en canvas sigue sirviendo para reducir los bytes de subida.

Infraestructura como código

CDK o SAM. CDK maneja mejor CloudFront con OAC; SAM es más liviano para API y Lambda. Dado que el frontend es JS vanilla sin build, CDK introduce una toolchain de Node que hoy no existe. La elección concreta se definirá al iniciar la Etapa 3.

Decisiones que sobreviven entre etapas

Decisión	Etapa 1	Etapa 2	Etapa 3
Modelo de producto (id uuid, priceMode)	✓	✓	✓
Repositorio async como costura	✓	✓	✓
Encuadre sin recorte (contain)	✓	✓	✓
Decisión de precio explícita	✓	✓	✓
localStorage	✓	✗	✗
Conservación del original	✗	Opcional	✓
Autenticación	✗	✗	✓
Manifiesto estático para lectura	✗	✓	✓

Trade-offs asumidos

Prototipo (Etapa 1)

- **Imágenes originales se pierden**: aceptado porque los datos son de prueba. La mitigación es no dejar que este esquema sobreviva a la iteración.

- **Cuota de localStorage**: con 10 productos el margen es amplio. Si falla, se captura el error y se informa.
- **Safari en navegación privada**: puede restringir el almacenamiento. Se detecta y se muestra un mensaje.

Publicación (Etapa 2)

- **El autor publica manualmente**: más pasos, pero sin exponer credenciales.
- **Sin autenticación**: cualquiera con acceso al editor local puede modificar.

Serverless (Etapa 3)

- **Costo por escritura**: Lambda y DynamoDB cobran por request. A este volumen es irrelevante.
- **Latencia de invalidación**: unos segundos entre la escritura y la propagación a CloudFront. Aceptable para un catálogo.

Riesgos y mitigaciones

Riesgo	Mitigación
Credenciales expuestas si el endpoint queda abierto	Cognito obligatorio desde el diseño. Validación en API Gateway.
Imágenes grandes bloquean el hilo principal	Procesamiento secuencial con cesión de control. Progreso visible.
Safari en modo privado bloquea storage	Detección temprana con mensaje claro.
El autor pierde datos del prototipo	Advertencia explícita de alcance. No se promete durabilidad.

Open questions

- **Forma de la plantilla de características**: ¿campos fijos (talla, color) o pares clave/valor libres?
- **Mecanismo de publicación en Etapa 2**: ¿exportar + subir manual, CLI, o anticipar Cognito?
- **Dominio propio**: ¿se usará un dominio custom con ACM y Route 53?
- **Región**: ¿cuál región de AWS para minimizar latencia desde Colombia?
- **Vista de detalle**: ¿el visitante puede ampliar la imagen? Si sí, S3 guarda varias resoluciones.

Conclusión

El diseño evoluciona en tres etapas que entregan valor incremental sin reescrituras:

1. **Prototipo local**: valida la interfaz con la menor infraestructura posible.
2. **Publicación estática**: hace visible el catálogo con un bucket S3 y CloudFront.
3. **Serverless completo**: autenticación, escritura protegida, y escala sin mantenimiento.

La costura de repositorio y el modelo de producto son las dos piezas que se diseñan una vez y sobreviven a todas las etapas. Todo lo demás (estilos, layout, almacenamiento) puede ajustarse o reemplazarse sin costo de retrofit.

Etapa 4: E-commerce con carrito y pagos

Objetivo

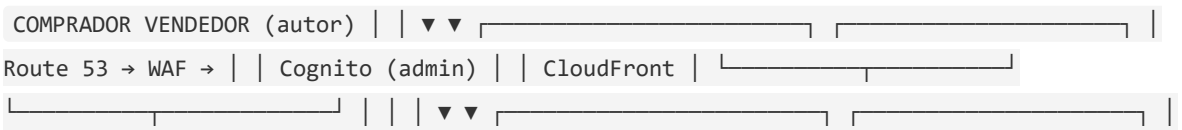
Convertir el catálogo en una tienda online funcional: carrito de compras, checkout con pago, gestión de stock real, y notificaciones al comprador. Mantener también el flujo de contacto directo (WhatsApp/email) para productos sin precio o consultas.

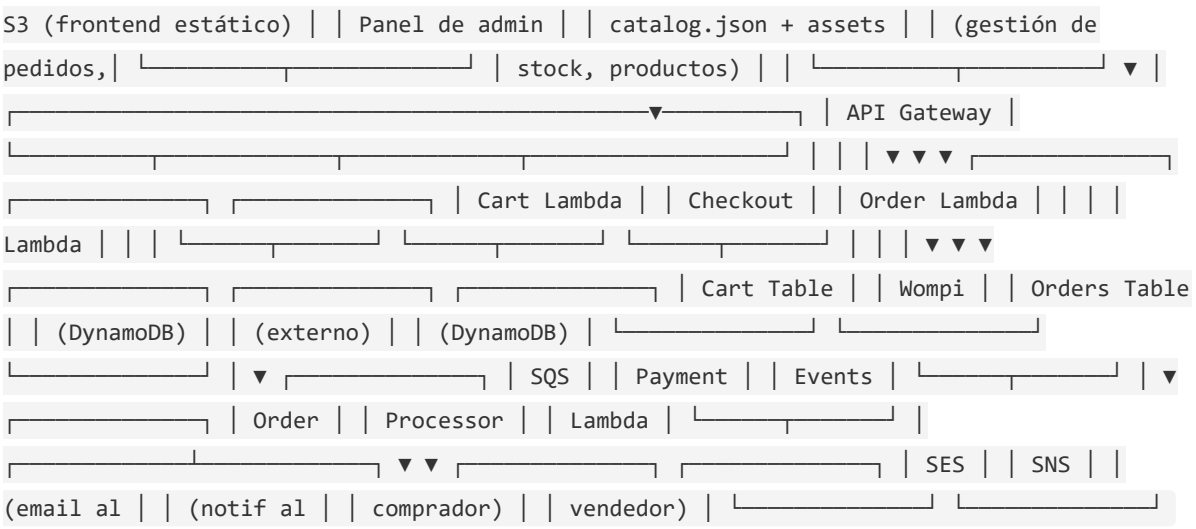
Decisiones de negocio

Aspecto	Decisión
Modelo de venta	Vendedor único
Flujos de compra	Pago online + contacto directo (WhatsApp/email)
Inventario	Stock real, se descuenta al confirmar pago
Pasarela de pago	**Wompi** (nativa COP, menor complejidad, PSE incluido)
Notificaciones	Email al comprador (confirmación, tracking)

Arquitectura adaptada de la referencia

La arquitectura de referencia de microservicios se simplifica para un vendedor único:





Componentes nuevos vs Etapa 3

Componente	Etapa 3	Etapa 4
WAF	No	Sí (protección de endpoints de pago)
Cart Service	No	Sí (Lambda + DynamoDB)
Checkout	No	Sí (Lambda + Wompi)
Order Service	No	Sí (Lambda + DynamoDB)
SQS	No	Sí (eventos de pago asincrónicos)
SES	No	Sí (emails transaccionales)
SNS	No	Sí (notificaciones al vendedor)
Secrets Manager	No	Sí (claves de Wompi)

Modelo de datos extendido

El producto de la Etapa 3 se extiende con campos de inventario:

``js Product { // Campos existentes (Etapa 3) id: string name: string priceMode: 'amount' | 'inquire' priceCOP: number | null imageRef: string createdAt: number

// Campos nuevos (Etapa 4) stock: number // cantidad disponible sku: string | null // código interno opcional active: boolean // visible en el catálogo } ``

Nuevas entidades:

``js CartItem { cartId: string // sessionId o visitorId productId: string quantity: number addedAt: number ttl: number // expiración automática }

Order { orderId: string status: 'pending' | 'paid' | 'shipped' | 'delivered' | 'cancelled' items: OrderItem[] totalCOP: number buyerEmail: string buyerPhone: string | null shippingAddr:

Address | null paymentRef: string | null // referencia de Wompi contactOnly: boolean // true si fue pedido por WhatsApp createdAt: number updatedAt: number } ``

Flujo de checkout con Wompi

`

1. Comprador llena carrito │ ▼
2. POST /checkout → Valida stock → Crea orden en estado 'pending' → Reserva stock temporalmente │ ▼
3. Redirect a Wompi (hosted checkout) → Wompi muestra formulario de pago → Comprador paga │ ▼
4. Webhook de Wompi → API Gateway → Lambda → Valida firma del webhook → Actualiza orden a 'paid' → Descuenta stock definitivo → Encola notificaciones │ ▼
5. SQS → Order Processor → SES: email de confirmación al comprador → SNS: notificación push/email al vendedor `

Flujo de contacto directo (WhatsApp/email)

Para productos con `priceMode: 'inquire'` o cuando el comprador prefiere no pagar online:

`

1. Comprador agrega al carrito │ ▼
2. En checkout, elige "Consultar por WhatsApp" │ ▼
3. POST /checkout?contact=whatsapp → Crea orden en estado 'pending' con contactOnly: true → Genera link de WhatsApp con resumen del pedido │ ▼
4. Redirect a WhatsApp → Vendedor recibe mensaje con productos y cantidades → Negociación directa `

Integración con Wompi

Wompi ofrece dos modos de integración:

Modo	Descripción	Complejidad	Recomendado
Hosted Checkout	Redirect a página de Wompi	Baja	Sí, para empezar
Widget embebido	Formulario dentro de tu página	Media	Después, si quieres más control
API directa	Tokenización + cargo desde backend	Alta	No necesario

Empezamos con **Hosted Checkout**: el comprador va a Wompi, paga, y vuelve. Un webhook confirma el pago.

Seguridad

- **WAF**: rate limiting en endpoints de checkout, bloqueo de IPs sospechosas.
- **Secrets Manager**: claves pública y privada de Wompi, nunca en código ni variables de entorno.
- **Validación de webhooks**: firma HMAC de Wompi verificada antes de procesar.
- **Reserva de stock con TTL**: si el pago no se completa en 15 minutos, el stock se libera.
- **Idempotencia**: el webhook puede llegar múltiples veces; se procesa solo una.

Notificaciones

Evento	Destinatario	Canal	Contenido
Orden creada	Comprador	Email (SES)	Resumen del pedido, link de pago si aplica
Pago confirmado	Comprador	Email (SES)	Confirmación, número de orden
Pago confirmado	Vendedor	SNS → Email/Push	Alerta de nueva venta
Pedido enviado	Comprador	Email (SES)	Tracking si existe

Gestión de stock

```
Producto con stock = 5 | Comprador A agrega 3 al carrito Comprador B agrega 4 al carrito | ▼ Stock disponible = 5 (no se reserva al agregar) | Comprador A inicia checkout | ▼ Reserva temporal: stock_disponible = 2, reservado_A = 3 TTL: 15 minutos | Comprador B inicia checkout | ▼ Error: solo hay 2 disponibles | Comprador A paga | ▼ Stock definitivo = 2, reserva liberada
```

El stock se reserva temporalmente al iniciar checkout, no al agregar al carrito. Esto evita bloquear inventario por carritos abandonados.

Costos estimados (mensual, bajo volumen)

Servicio	Uso estimado	Costo
Lambda	10,000 invocaciones	~ (capa gratuita)
DynamoDB	1 GB, 25 WCU/RCU	~ (capa gratuita)
API Gateway	1M requests	~
SES	1,000 emails	~.10
SNS	1,000 notificaciones	~ (capa gratuita)
SQS	1,000 mensajes	~ (capa gratuita)
WAF	1M requests	~
Secrets Manager	2 secretos	~.80
Wompi	2.79% + por transacción	Variable

Total fijo: **~/mes** + comisión de Wompi por venta.

Roadmap completo actualizado

Etapa	Objetivo	Almacenamiento	Auth	E-commerce
1	Prototipo local	localStorage	No	No
2	Publicación estática	S3 + CloudFront	No	No
3	Serverless (autor)	DynamoDB + S3	Cognito	No
4	E-commerce completo	DynamoDB + S3	Cognito	Sí (Wompi)

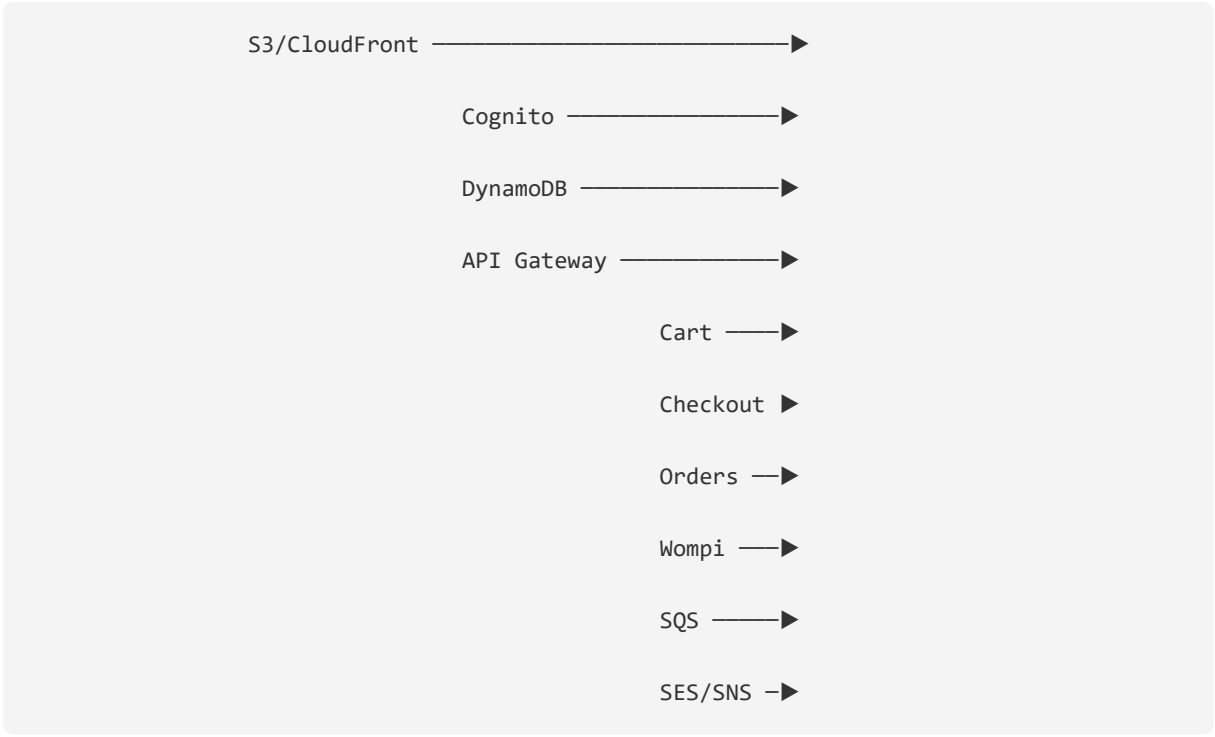
Dependencias entre etapas

` Etapa 1 Etapa 2 Etapa 3 Etapa 4 ═══════ ═══════ ═══════ ═══════

Modelo de producto ──▶

Repositorio async ──▶

Frontend estático ──▶



`

Open questions para Etapa 4

- **Envío**: ¿se integra con alguna transportadora, o el vendedor gestiona manualmente?
- **Impuestos**: ¿IVA incluido en el precio o calculado en checkout?
- **Devoluciones**: ¿política de reembolso? Wompi tiene API de reversión.
- **Múltiples métodos de pago**: Wompi soporta tarjeta, PSE, Nequi. ¿Todos habilitados?
- **Límite de carrito**: ¿máximo de items o monto?

Decisiones de pasarela de pago

Análisis comparativo para Colombia

Pasarela	Complejidad	Costo/transacción	COP nativo	PSE	Observaciones
Stripe	Media	2.9% + .30 USD	Sí (conversión)	No	Mejor API, pero conversión de moneda
Wompi	Baja	2.79% + COP	Sí	Sí	Recomendado para Colombia
PayU	Media-Alta	2.99% + IVA	Sí	Sí	Integración más verbosa
MercadoPago	Baja	3.49% + IVA	Sí	No	Fee más alto, retención de fondos

Decisión: Wompi

Razones:

1. **Menor complejidad**: Hosted Checkout funciona con redirect + webhook.
2. **COP nativo**: Sin conversión de moneda, pricing predecible.
3. **PSE incluido**: Transferencia bancaria muy usada en Colombia.
4. **Costo competitivo**: 2.79% es el más bajo de la lista.
5. **Soporte local**: Bancolombia, documentación en español.
6. **Sandbox gratuito**: Desarrollo sin costo.

Stripe queda como alternativa si en el futuro se necesita venta internacional.